DFSha: sistema de archivos distribuido con alta disponibilidad

Propuesta técnica individual · Fase I · Opción 1: cliente/servidor con servicio distribuido S2S

Abel Santiago Garcia Ortiz · Sistemas Distribuidos · 2026-2
Profesor: Álvaro Ospina · Universidad Pontificia Bolivariana

Contenido

1 Introducción, alcance y supuestos 1
2 Definición del servicio y casos de uso 1
3 Arquitectura 3
4 Distribución de datos, metadatos y réplicas 5
5 Protocolos e interfaces 5
6 Semántica de acceso, consistencia y concurrencia 6
7 Tolerancia a fallos y disponibilidad 7
8 Escenarios obligatorios 8
9 Seguridad 9
10 Métricas, experimentos y compromisos 10
Referencias 11
Anexos 12

1 Introducción, alcance y supuestos

Este documento propone el diseño de DFSha, un sistema de archivos distribuido que reparte archivos grandes en bloques replicados entre varios nodos, con alta disponibilidad, consistencia comprobable y seguridad. Sigue la opción 1, asignada por el profesor: un servicio cliente/servidor cuyos servidores cooperan entre sí (S2S) en una red privada.

La idea central es que **los bloques nunca se modifican y cada archivo cambia de versión con un único commit atómico** de sus metadatos: así las réplicas no pueden divergir, nunca se publica un archivo incompleto y recuperar una escritura interrumpida se reduce a recolectar bloques sin referencia. El diseño separa metadatos y datos como GFS [1] y HDFS [2], toma de AFS [6] la semántica close-to-open, de Chubby [4] y de Gray y Cheriton [5] los locks con lease, y usa Raft [3] a través de etcd.

Escala objetivo (D20): hasta 10 nodos de datos, 100 usuarios, archivos de hasta 10 GiB y menos de 1 TiB en total, en una zona de disponibilidad, como en un despliegue en AWS Academy. Cada simplificación que esto permite declara su límite y su extensión.

Supuestos: fallos por detención, no bizantinos; relojes con deriva acotada (los leases usan duraciones locales y los fencing tokens cubren las pausas); red privada pero no confiable; discos que pueden corromper datos; servidores y operadores de confianza.

Limitaciones: no se comparten archivos entre usuarios; no hay semántica POSIX completa (las escrituras se ven tras `flush` o `close` y no se admiten huecos después del final); no hay versiones históricas y la lectura estable de una versión reemplazada dura al menos una hora; no se rebalancean los bloques existentes; sin quórum de etcd se rechazan las operaciones de metadatos. Estas simplificaciones de RF3 y de las garantías quedan sujetas a la aprobación del profesor antes de la implementación.

Las decisiones de diseño se citan como D01–D20 y se resumen en el Anexo D.

2 Definición del servicio y casos de uso

DFSha ofrece a cada usuario autenticado un sistema de archivos jerárquico privado, accesible con la CLI `dfs` o con una librería Python que no conocen direcciones de nodos (RNF7). Cada usuario ve su espacio como la raíz `/`, que internamente es un directorio raíz propio (D15): no existe una ruta que nombre archivos ajenos (RF1, RNF6).

Actores. El *usuario* gestiona sus archivos (RF1–RF3). El *operador* da de alta usuarios, emite los certificados de los nodos y consulta el estado del clúster. El *coordinador*, actor de sistema, es el nodo `meta` elegido (D02) que dispara la re-replicación y la limpieza de bloques.

2.1 Operaciones

RF	Operaciones	Semántica
RF1	`ls`, `cd`, `pwd`, `stat`	Listar un directorio y consultar atributos (versión, tamaño de bloque, replicación). `cd` y `pwd` son estado local del cliente.
RF1	`mkdir [-p]`, `rmdir`, `rm [-r]`, `mv`	Crear directorios, borrar uno vacío (`NOT_EMPTY` si no), borrar un archivo o un subárbol, y renombrar o mover; cada una es una transacción atómica (D15).
RF2	`put`, `get`	Cargar (crear o reemplazar) o descargar un archivo completo; `put` publica la versión de una vez y `get` verifica cada hash.
RF3	`open(ruta, modo)`	Modos `r`, `w` (crear o truncar), `a` (anexar) y `r+` (leer y modificar); el descriptor queda ligado a una versión.
RF3	`read`, `write`	Con offset explícito; `read` lee la versión fijada y `write` escribe en la sesión (en modo `a`, al final).
RF3	`flush`, `close`	Publican los cambios pendientes como versión nueva; `close` además libera locks y sesión.
RF3	`lock(rango, espera)`, `unlock`	Lock exclusivo con lease sobre un rango de bloques; con `espera`, la librería reintenta (D07, D17).

Semántica declarada de write(). Admite **creación** y **reemplazo** (modo `w` o `put`), **anexado** (modo `a`) y **modificación parcial** (modo `r+`, en cualquier offset hasta el tamaño actual, sin huecos). La unidad interna es el bloque de 8 MiB: modificar un byte reescribe ese bloque como uno nuevo (D03). Los cambios se ven tras `flush` o `close` (close-to-open, D06); la sección 6 formaliza las garantías.

2.2 Casos de uso

ID	Caso	Actor	Flujo principal
CU-01	Iniciar sesión	Usuario	`meta` verifica la contraseña (Argon2id) y devuelve un JWT de una hora
CU-02	Gestionar directorios	Usuario	Operaciones de RF1; solo involucran a `meta`
CU-03	Cargar archivo	Usuario	`put`: reservar bloques, replicarlos en cadena y confirmar (figura)
CU-04	Leer completo o un rango	Usuario	`get`, o `open(r)` + `read`: solo los bloques necesarios, en paralelo
CU-05	Modificar o anexar	Usuario	`open(r+)` u `open(a)` + `write` + `close`: se reescriben los bloques afectados y se confirma una versión nueva
CU-06	Borrar	Usuario	`rm` o `rmdir`: desvinculación atómica; el espacio se libera después
CU-07	Administrar	Operador	Alta y baja de usuarios; estado del clúster
CU-08	Mantener réplicas	Coordinador	Re-replicar los bloques de nodos caídos y borrar bloques sin referencia

Los errores de cada operación se detallan en el Anexo A.

2.3 Secuencias principales

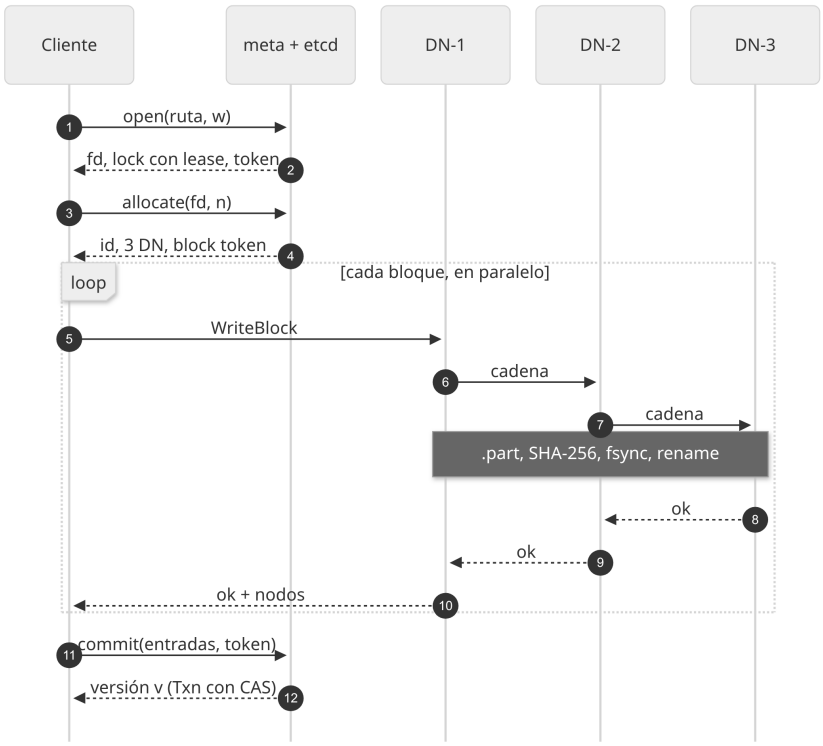


Figura 1. *Secuencia de carga (`put`). Las flechas punteadas son respuestas; varios bloques viajan en paralelo.*

La figura muestra la **carga**; el Anexo E, la lectura, la escritura parcial y el borrado. Una **escritura parcial** sigue el mismo flujo, pero con un lock solo sobre los bloques afectados, que se reescriben como bloques nuevos (sección 6). En una **lectura**, `open(r)` fija la versión y el cliente lee cada bloque de una réplica, verificando su hash (sección 4). En un **borrado**, `rm` desvincula la entrada en una transacción; quien ya tenía el archivo abierto sigue leyendo durante el período de gracia (D18) y luego el coordinador libera los bloques.

3 Arquitectura

DFSha separa un **plano de control**, que guarda y coordina los metadatos, de un **plano de datos**, que almacena y transfiere bloques. El contenido de los archivos nunca pasa por el plano de control, así que el ancho de banda total crece con los nodos de datos (RNF5).

3.1 Componentes

Componente	Instancias	Responsabilidades
Cliente `dfs` (CLI Typer + librería)	uno por usuario	Divide y reconstruye archivos, transfiere bloques en paralelo, verifica hashes, mantiene la sesión y guarda ubicaciones en caché.
`meta` (FastAPI + grpcio)	3, sin estado	API REST para clientes y gRPC para los DN: autentica, resuelve rutas, atiende `open`, `allocate`, `commit` y `lock` y elige réplicas. Su estado vive en etcd, así que cualquier instancia atiende cualquier petición (D01).
etcd	3, Raft	Almacén replicado y linealizable de todos los metadatos, con transacciones con comparación, leases y `watch`.
Coordinador	rol de un `meta`	Elegido con un lease (D02): vigila los DN, re-replica y limpia bloques.

Componente	Instancias	Responsabilidades
Nodo de datos (DN)	$N \geq 4$	Guarda bloques cifrados, los reenvía en cadena y los sirve; verifica *block tokens*; envía heartbeats y reportes de bloques.

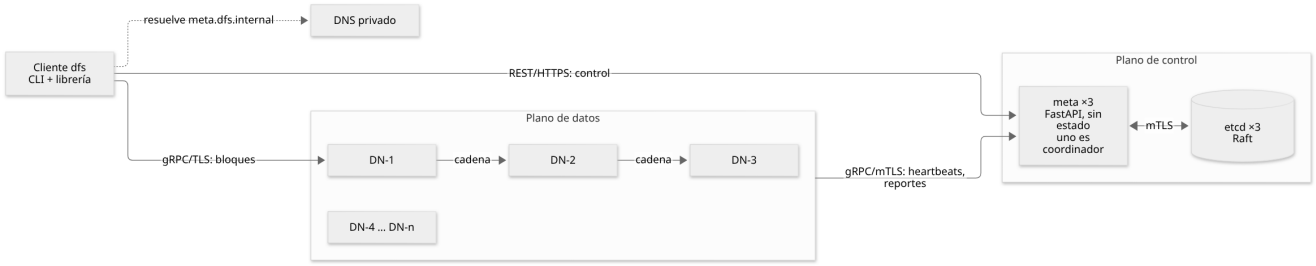


Figura 2. *Componentes y planos de DFSha. Las flechas indican quién inicia cada comunicación.*

Vivacidad de los nodos (D11). Cada DN envía un heartbeat cada 5 s a cualquier `meta`, que renueva la clave `/alive/<dn>` (lease de 15 s, los mismos plazos de las sesiones) y responde con las órdenes pendientes del nodo. Tras tres heartbeats perdidos la clave vence y el nodo pasa a *sospechoso*: `meta` deja de incluirlo en cadenas nuevas. El coordinador vigila esas claves con `watch` y lo declara caído si no vuelve en 60 s, para no re-replicar por un simple reinicio (sección 7). Así los DN no necesitan conocer al coordinador, y cambiarlo no pierde estado.

3.2 Despliegue

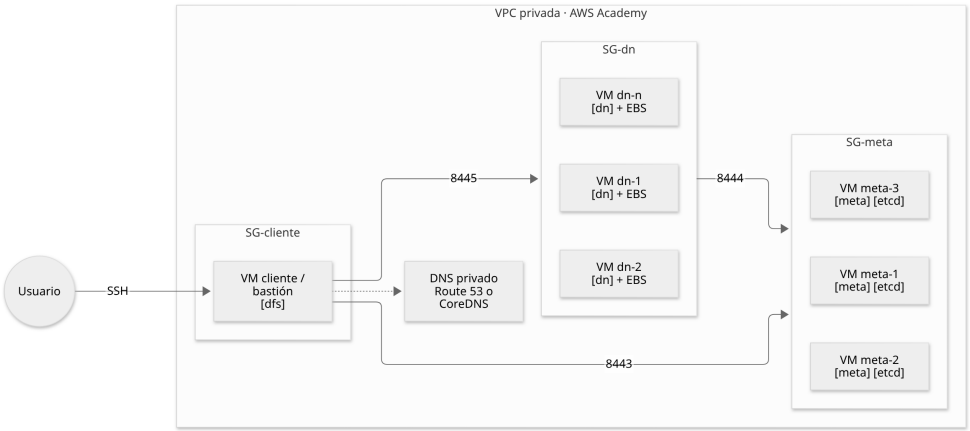


Figura 3. *Despliegue en AWS Academy. Cada caja interior es un contenedor Docker; cada VM pertenece a un grupo de seguridad.*

Cada componente corre en un contenedor Docker sobre EC2, en una VPC privada: tres VM con `meta` y etcd, al menos cuatro VM de datos con un volumen EBS y una VM cliente que también es el bastión, la única con SSH desde fuera. `meta.dfs.internal`, el único nombre que conocen clientes y DN (D09, RNF7), se publica en una zona privada de Route 53 o en un contenedor CoreDNS. Los grupos de seguridad solo admiten los flujos del Anexo C, y `meta` nunca inicia conexiones hacia los DN: sus órdenes viajan en las respuestas a los heartbeats.

3.3 Escalabilidad

`meta` no tiene estado y se replica bajo el mismo nombre DNS; el techo es el grupo Raft de etcd, que podría repartirse por usuario entre varios clústeres porque ninguna operación cruza espacios de usuarios. A la escala objetivo (D20) los metadatos ocupan decenas de megabytes y los manifiestos admiten archivos de hasta unos 90 GiB (D13). Un DN nuevo se registra solo y entra de inmediato en la colocación, y como `meta` no está en el camino de los datos, el throughput agregado crece con los DN (RNF1).

4 Distribución de datos, metadatos y réplicas

4.1 Bloques y metadatos

Un archivo se divide en bloques de 8 MiB por defecto (D04); el último puede ser menor. Cada bloque tiene un identificador aleatorio de 128 bits, que no se deriva del contenido para no revelar si dos usuarios guardan los mismos datos, y un SHA-256 del contenido sin cifrar. Ningún bloque se modifica después de escrito (D03).

Todos los metadatos viven en etcd (Anexo B). Un archivo vigente es la cadena `dirent → inode → manifest → blocks`: la entrada de directorio apunta al inodo, el inodo a su manifiesto vigente, el manifiesto lista los bloques `(block_id, tamaño, sha256)` y `/blocks/<id>` guarda las réplicas confirmadas de cada uno. Los manifiestos son inmutables y ocupan unos 130 bytes por bloque: el de un archivo de 10 GiB pesa unos 170 KB y cabe en un valor de etcd (D13). Al reemplazar o borrar un archivo, su manifiesto anterior queda *retirado* durante una hora para los lectores que aún lo usan (D18).

Un commit escribe primero, en lotes, las entradas `/blocks` de los bloques nuevos, que nadie referencia todavía; después, una única transacción guarda el manifiesto nuevo, apunta el inodo a él y retira el anterior. Si la transacción falla, lo escrito antes queda sin referencia y lo limpia el GC (sección 6).

4.2 Colocación de réplicas

A diferencia del cálculo determinista de Ceph [7], la colocación se decide en `meta` y queda registrada. Para cada bloque, `meta` elige tres DN vivos con la técnica de las *dos opciones* [10]: dos candidatos al azar, de los que gana el de más espacio libre (D05). Cada bloque se coloca por separado, así que un archivo queda repartido entre todos los nodos (RNF4) y la carga se equilibra sin estado global. Con solo dos DN vivos el bloque queda sub-replicado; con menos, `allocate` responde `UNAVAILABLE`.

4.3 Confirmaciones y ubicaciones

Un DN confirma un bloque solo después de persistirlo, y la confirmación sube por la cadena hasta el cliente con la lista de nodos que lo guardaron. En el commit, `meta` exige al menos dos de esos nodos vivos por bloque y los registra en `/blocks` (D16); puede confiar en el cliente porque los archivos no se comparten, así que un cliente que mintiera solo dañaría sus propios datos. Después del commit, solo el coordinador cambia ubicaciones: agrega réplicas al re-replicar y retira las de un nodo caído, cuyos bloques encuentra recorriendo `/blocks`.

4.4 Reconstrucción

El cliente pide las ubicaciones de los bloques del rango que necesita, descarga hasta ocho en paralelo de las réplicas con menos fallos recientes, verifica cada SHA-256 y escribe cada bloque en su posición, sin importar el orden de llegada. `put` registra además el SHA-256 del archivo completo, que `get` verifica: es la evidencia de "archivo recuperado idéntico" de la prueba de carga.

5 Protocolos e interfaces

Interfaz	Protocolo	Puerto	Autenticación	Por qué
Cliente → `meta`	REST/JSON sobre HTTPS (FastAPI)	8443	JWT del usuario	Control: poco volumen y fácil de probar; FastAPI publica la especificación OpenAPI y valida con Pydantic.
Cliente → DN, DN → DN	gRPC con streaming (`grpcio`)	8445	TLS + *block token*; mTLS entre DN	Bloques en fragmentos de 1 MiB con el control de flujo de HTTP/2; el mismo servicio sirve al cliente y a la cadena.

Interfaz	Protocolo	Puerto	Autenticación	Por qué
DN → `meta`	gRPC (`grpcio`)	8444	mTLS	Registro, heartbeats y reportes; las órdenes vuelven en la respuesta.
`meta` → etcd	API v3 de etcd (`etcd3gw`)	2379	mTLS	Transacciones, leases y `watch`.

No se usa cola de mensajes: ningún flujo necesita entrega diferida ni difusión, y las órdenes del coordinador viajan en las respuestas a los heartbeats (D08). Si un `meta` no responde o no tiene quórum, el cliente prueba otra dirección de `meta.dfs.internal` (D09).

Errores. Toda respuesta de error REST tiene la forma del ejemplo y gRPC usa el estado equivalente; los códigos principales son `UNAUTHENTICATED` (401), `PERMISSION_DENIED` (403), `NOT_FOUND` (404), `CONFLICT` y `STALE_LEASE` (409), `INSUFFICIENT_REPLICAS` (422), `LOCKED` (423), `UNAVAILABLE` (503) y `NO_SPACE` (507). El Anexo A especifica endpoints, servicios gRPC, ejemplos y la acción del cliente ante cada error.

```
423 Locked
{ "error": { "code": "LOCKED", "message": "bloques 2-5 con lock de otra sesión",
             "details": { "from": 2, "to": 5, "retry_after_ms": 4000 } } }
```

6 Semántica de acceso, consistencia y concurrencia

6.1 Garantías

Los metadatos se leen y escriben en etcd con operaciones linealizables (RNF3). Una escritura está **confirmada** cuando `commit` responde `200` con una versión `v`; desde ese momento:

- **G1. Visibilidad:** todo `open` posterior, desde cualquier cliente y contra cualquier `meta`, ve `v` o una versión más nueva.
- **G2. Durabilidad:** cada bloque de `v` está sincronizado a disco en al menos dos DN.
- **G3. Atomicidad:** un lector ve todos los cambios de un commit o ninguno.
- **G4. Lectura estable:** un descriptor abierto sobre `v` lee `v` aunque se publiquen versiones nuevas o se borre el archivo, durante al menos una hora (D18).
- **G5. Lee lo propio:** quien escribe ve sus cambios aún no publicados en su propio descriptor.
- **G6. Exclusión:** dos commits sobre el mismo bloque nunca se aplican ambos sobre la misma versión base; uno recibe `LOCKED`, `CONFLICT` o `STALE_LEASE`.

G4 y G5 son, dentro de un descriptor, las lecturas monotónicas y *read-your-writes* de los modelos centrados en el cliente [9]. Un descriptor ya abierto no ve versiones nuevas: debe reabrir. La sección 10 prueba cada garantía.

6.2 Sesiones y locks

Cada descriptor es una **sesión**: un lease de etcd de 15 s que la librería renueva cada 5 s y al que se atan sus locks, así que si el cliente muere se liberan solos.

Un lock es una clave `/locks/<ino>/<sesión>/<n>` con un rango de bloques, siempre exclusivo, y su `create_revision` en etcd es su **fencing token**, porque las revisiones solo crecen [8]. Tras crear la clave, `meta` busca claves anteriores del inodo con un rango solapado: si no hay, concede el lock; si hay, borra la suya y responde `LOCKED`. Como gana la clave más antigua, dos peticiones simultáneas nunca obtienen ambas el lock. Con `espera`, la librería reintenta con espera exponencial (D17), sin orden justo entre quienes esperan.

La librería toma los locks sola (D19): `open(w)` sobre todo el archivo, `open(a)` desde el último bloque y, en `r+`, cada `write` sobre los bloques que toca. La respuesta del lock incluye las entradas vigentes del rango, así que una modificación parcial se aplica sobre el contenido más reciente aunque el descriptor se haya abierto en una versión anterior. Los lectores no toman locks: les basta G4.

6.3 Commit

1. `meta` verifica la sesión y que cada bloque nuevo tenga al menos dos réplicas confirmadas en nodos vivos (`INSUFFICIENT_REPLICAS` si no), y escribe sus entradas `/blocks`.
2. Una transacción compara que la `create_revision` de cada lock usado sea su fencing token (si no, `STALE_LEASE`), que el inodo siga en la versión leída y que `/commits/<commit_id>` no exista. Si todo coincide, guarda el manifiesto nuevo, apunta el inodo a él, retira el anterior y registra el resultado en `/commits/<commit_id>`.
3. Si el inodo avanzó por un commit ajeno sobre bloques disjuntos, `meta` rehace el manifiesto sobre la versión nueva y repite el paso 2; si un bloque modificado cambió desde la base del escritor, responde `CONFLICT`, que con locks es solo una red de seguridad.

Si la respuesta se pierde, reintentar con el mismo `commit_id` devuelve la misma versión: el commit es idempotente.

6.4 Integridad y escrituras interrumpidas

El SHA-256 de cada bloque se verifica de extremo a extremo: lo calcula el cliente, el DN lo comprueba antes de confirmar y el lector al leer; en disco, la etiqueta de AES-GCM detecta alteraciones. El DN escribe cada bloque en `<id>.part`, lo sincroniza, lo renombra a `<id>`, sincroniza el directorio y solo entonces confirma; al arrancar borra los `.part` que encuentre.

Si un escritor muere antes del commit, su sesión vence, el archivo sigue en la última versión confirmada y el GC borra los bloques ya subidos (D18). Si cae un `meta` o el líder de etcd durante el commit, la transacción se aplicó entera o no se aplicó, y el reintento idempotente lo resuelve.

La integridad no sustituye la coordinación: los hashes prueban que un bloque no se corrompió, pero dos escrituras válidas pueden pisarse; eso lo impiden los locks, los fencing tokens y la comparación de versiones del commit.

7 Tolerancia a fallos y disponibilidad

Fallo	Detección	Efecto visible	Recuperación
DN	`/alive/<dn>` vence: sospechoso a los 15 s, caído a los 60 s	Se lee de otra réplica; queda fuera de las cadenas nuevas	Re-replicación de sus bloques
DN en una cadena de escritura	Error en el stream	El `put` tarda algo más	Con dos copias se acepta sub-replicado; si no, otra cadena (D19)
Instancia `meta`	Conexión fallida	Ninguno	Otra dirección del DNS; commit reintentado con el mismo `commit_id`
Coordinador	Vence su lease	Tareas de fondo en pausa unos segundos	Otro `meta` gana la elección y retoma desde etcd

Fallo	Detección	Efecto visible	Recuperación
Miembros de etcd	Raft	Con 1 de 3 caído, ninguno; con 2, o `meta` en minoría, `UNAVAILABLE` en metadatos y solo siguen los descriptores abiertos	Automática al recuperar el quórum
Cliente	Vence su sesión (15 s)	Locks liberados; cambios sin publicar descartados	GC de sus bloques (D18)
Corrupción en disco	Hash al leer	El lector usa otra réplica	Re-replicación desde una réplica sana

Re-replicación. El coordinador atiende primero los bloques con menos réplicas: envía a un destino, en la respuesta a su heartbeat, una orden `Replicate`; el destino copia el bloque de una réplica viva, lo verifica e informa el resultado. Cada DN hace como máximo dos copias a la vez, y el coordinador solo escribe órdenes mientras `/coordinator` sea suya, así que uno destituido no puede emitirlas.

Nodo que vuelve. Antes de 60 s no hace falta nada, porque sus bloques nunca cambian; si ya se lo declaró caído, su reporte de bloques se concilia y se borran las copias sobrantes.

Disponibilidad (RNF2). Leer exige quórum de etcd para abrir y una réplica viva por bloque, así que tolera dos DN caídos; escribir exige quórum y dos DN vivos. Ante una partición, los metadatos priorizan la consistencia (D12): el lado sin quórum rechaza operaciones en lugar de divergir.

8 Escenarios obligatorios

8.1 Dos procesos escriben el mismo bloque simultáneamente

P1 y P2 abren el archivo en `r+` sobre la versión 6 y escriben en el bloque 3, así que sus librerías piden un lock sobre él. etcd ordena las claves (revisión 48213 para P1, 48214 para P2): P1 obtiene el lock y P2 recibe `LOCKED` y reintenta. P1 confirma la versión 7 y cierra; en un reintento, P2 obtiene el lock con la entrada vigente del bloque, aplica su escritura sobre la versión 7 y confirma la 8. **Resultado:** las escrituras se serializan, prevalecen los bytes de P2 donde se solapan y no se pierde la de P1.

Si P1 se detiene más de 15 s, su lock vence y P2 lo obtiene (figura). Cuando P1 despierta e intenta publicar con el token 48213, el commit falla con `STALE_LEASE`: un escritor que perdió el lock no puede publicar aunque crea tenerlo [8].

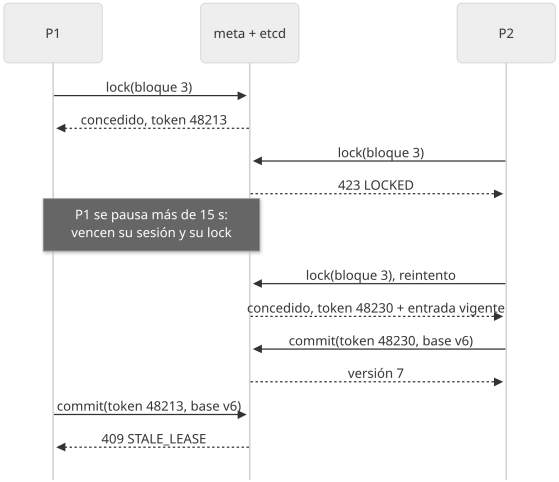


Figura 4. *Escenario 1 con pausa de P1: el fencing token impide que publique un escritor cuyo lock venció.*

8.2 Una réplica está desconectada durante una escritura

El bloque 3 se asignó a la cadena DN-2 → DN-4 → DN-1 y DN-4 está desconectado. DN-2 guarda su copia pero no puede reenviarla, así que el cliente recibe un solo nodo guardado; como no alcanza W = 2, pide otra cadena sin DN-4 y reenvía el bloque con un `block_id` nuevo, y el GC borra la copia huérfana de DN-2. Si DN-4 hubiera sido el último de la cadena, el bloque se habría aceptado con dos réplicas y el coordinador habría creado la tercera.

Qué observan los lectores: antes del commit, la versión anterior completa; después, la nueva, cuyas ubicaciones solo incluyen nodos que confirmaron el bloque. Ningún lector es dirigido a DN-4 para ese bloque, y los bloques antiguos que DN-4 guarda siguen siendo válidos porque nunca cambian. Al volver, DN-4 descarta su `.part` incompleto (sección 7).

8.3 Un nodo falla después de recibir parte de un bloque

Tres barreras impiden publicar un archivo incompleto: (1) el DN guarda los datos parciales solo en `<id>.part` y confirma después de recibir, verificar, sincronizar y renombrar el bloque completo, así que un nodo que cae a mitad nunca confirma; (2) el commit exige dos réplicas confirmadas por bloque, y si la cadena no las alcanzó el cliente reintenta el bloque con otra; (3) el archivo solo cambia de versión con la transacción final. Un lector ve la versión anterior o la nueva, nunca una intermedia (G3), y lo escrito sin confirmar lo borra el GC.

8.4 El cliente desconoce la ubicación física de los bloques

Lo único que el cliente trae configurado es el nombre `meta.dfs.internal` y la CA. El DNS devuelve las tres direcciones de `meta`, y el cliente prueba otra si una falla. Tras `login` y `open`, las ubicaciones llegan bajo demanda: `GET /sessions/{fd}/blocks` devuelve, para un rango de bloques, sus réplicas confirmadas con dirección y token, y `allocate` devuelve la cadena para escribir. El cliente las guarda en caché mientras dure su token: como los bloques no cambian, una entrada solo queda obsoleta si su réplica cae, y el primer error la invalida; como los DN se registran solos, agregar o perder nodos no requiere cambiar nada en el cliente (RNF7).

9 Seguridad

Amenazas consideradas: un atacante en la red privada que escucha o altera tráfico, un usuario que intenta acceder a archivos ajenos, un token filtrado y un disco sustraído. **Fuera de alcance:** servidores comprometidos, operadores maliciosos y denegación de servicio.

9.1 Autenticación y autorización

Las contraseñas se guardan con Argon2id [12]. `login` devuelve un JWT [14] válido por una hora, con los claims `sub`, `role` y `exp`, firmado con Ed25519 por una llave que comparten las tres instancias de `meta`; los intentos fallidos se limitan. Cada nodo tiene un certificado de una CA privada que indica su rol (p. ej. `dn-3.dn.dfs.internal`), y cada servicio solo acepta los roles que le corresponden: `ControlService`, a los DN; los reenvíos, a otros DN; etcd, a `meta`.

Toda ruta se resuelve desde la raíz del solicitante (D15), así que no se puede nombrar un archivo ajeno; además, `meta` compara el dueño de cada inodo con el `sub` del token, y el rol `admin` habilita CU-07. Tras autorizar una operación, `meta` emite *block tokens*: JWT de 10 minutos con usuario, bloque y operación, que el DN verifica localmente. Un token filtrado solo sirve para ese bloque, esa operación y por pocos minutos.

9.2 Cifrado

En tránsito: TLS 1.3 [11] en todo el tráfico: HTTPS entre cliente y `meta`, TLS entre cliente y DN, y mTLS en las conexiones internas (DN → `meta`, DN → DN, `meta` → etcd y entre miembros de etcd). El cliente verifica los certificados contra la CA privada, y ningún puerto acepta texto en claro.

En reposo: cada DN cifra cada bloque con AES-256-GCM [13], con una llave propia del bloque derivada por HKDF de la llave maestra del nodo y del `block_id`, y un nonce aleatorio; el `block_id` va además como dato autenticado, así que alterar o intercambiar archivos en disco se detecta. Como cada bloque se cifra una sola vez, ninguna llave se usa con dos nonces. La llave maestra es un secreto de Docker; etcd y los secretos residen en volúmenes EBS cifrados (o LUKS), y las contraseñas solo existen como hashes.

9.3 Protección de la red

La VPC solo admite SSH entrante al bastión, y los grupos de seguridad solo los flujos del Anexo C: los clientes no alcanzan etcd y `meta` nunca conecta hacia los DN. Los accesos denegados se registran como evidencia para la prueba de seguridad.

10 Métricas, experimentos y compromisos

10.1 Métricas

Se miden la latencia (p50, p95 y p99) de `open`, `commit`, `put` y `get`; el throughput por cliente y agregado; los commits por segundo y la tasa de `LOCKED` sobre un mismo archivo; y, durante fallos, la fracción de operaciones exitosas y los tiempos de detección y de recuperación de R = 3. `meta` y los DN exponen métricas Prometheus, y un cliente de pruebas repite cada experimento al menos 5 veces con archivos de SHA-256 conocido.

10.2 Experimentos previstos

ID	Experimento	Condiciones	Hipótesis	Cubre
E1	Carga y recuperación	Archivos de 10 MB a 4 GB	Hash idéntico; throughput limitado por la red del cliente	Carga
E2	Semántica de RF3	Modos de `write`; lector abierto mientras otro publica	Lo declarado; G4 y G5	RF3
E3	Replicación	R = 1 frente a R = 3	La cadena agrega latencia moderada al `put`; `get` no cambia	Rendimiento
E4	Concurrencia	1 a 8 clientes: mismo bloque, bloques o archivos distintos; escritor pausado con `SIGSTOP`	Mismo bloque serializado y `STALE_LEASE` al pausado (G6); el resto escala	Concurrencia
E5	Escalabilidad	1 a 8 lectores con 4 y 6 DN	El throughput agregado crece con los lectores y con los DN	Rendimiento
E6	Caída de nodos	Un DN, un `meta`, el coordinador, uno y dos miembros de etcd	Sin errores visibles (G2); sin quórum, `UNAVAILABLE`	Fallos
E7	Consistencia	DN aislado o cliente detenido a mitad de un `put`	Nunca una versión parcial (G3); G1	Consistencia
E8	Seguridad	Tokens ajenos o vencidos; captura de tráfico; inspección de discos	401 o 403; nada en claro	Seguridad

10.3 Compromisos técnicos

Decisión	Gana	Cuesta
Bloques inmutables de 8 MiB (D03, D04)	Consistencia y recuperación simples; paralelismo	Modificar un byte reescribe 8 MiB; requiere GC

Decisión	Gana	Cuesta
Cadena con W = 2 de R = 3 (D05)	Ancho de banda del cliente; escrituras con un DN caído	Algo de latencia; ventana con dos copias
Metadatos CP en etcd (D01, D12)	Consistencia fuerte sin consenso propio	Sin quórum no hay operaciones; un grupo Raft como techo
Close-to-open (D06)	Semántica simple y comprobable	Escrituras ajenas visibles solo al reabrir
Locks con lease (D07)	Se liberan solos ante fallos	Una pausa de más de 15 s pierde el lock; el fencing token lo hace seguro

Justificación tecnológica. FastAPI genera la especificación OpenAPI y valida cada petición; `grpcio` ofrece streaming asíncrono; etcd reúne Raft, transacciones con comparación, leases y `watch`, es más simple de operar que ZooKeeper y evita implementar consenso propio. Todo corre en Docker sobre Linux. Se descartó Go para alinearse con las herramientas del curso, aceptando un rendimiento menor que E3 y E5 medirán (D14).

Referencias

1. S. Ghemawat, H. Gobioff y S.-T. Leung, “The Google File System”, *SOSP*, 2003.
2. K. Shvachko, H. Kuang, S. Radia y R. Chansler, “The Hadoop Distributed File System”, *IEEE MSST*, 2010.
3. D. Ongaro y J. Ousterhout, “In Search of an Understandable Consensus Algorithm”, *USENIX ATC*, 2014.
4. M. Burrows, “The Chubby Lock Service for Loosely-Coupled Distributed Systems”, *OSDI*, 2006.
5. C. Gray y D. Cheriton, “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency”, *SOSP*, 1989.
6. J. H. Howard *et al.*, “Scale and Performance in a Distributed File System”, *ACM TOCS*, vol. 6, n.º 1, 1988.
7. S. A. Weil *et al.*, “Ceph: A Scalable, High-Performance Distributed File System”, *OSDI*, 2006.
8. M. Kleppmann, *Designing Data-Intensive Applications*, O’Reilly, 2017.
9. M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.ª ed., 2023.
10. M. Mitzenmacher, “The Power of Two Choices in Randomized Load Balancing”, *IEEE Transactions on Parallel and Distributed Systems*, vol. 12, n.º 10, 2001.
11. E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3”, RFC 8446, IETF, 2018.
12. A. Biryukov, D. Dinu, D. Khovratovich y S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications”, RFC 9106, IETF, 2021.
13. M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC”, NIST SP 800-38D, 2007.
14. M. Jones, J. Bradley y N. Sakimura, “JSON Web Token (JWT)”, RFC 7519, IETF, 2015.

Anexos

Anexo A. Especificación de APIs

Convenciones

- Base: `https://meta.dfs.internal:8443/v1`. Cuerpos JSON en UTF-8. Toda petición, salvo `login`, lleva `Authorization: Bearer <JWT>`.
- `{path}` es una ruta relativa a la raíz del usuario, codificada en la URL; puede contener `/`.
- `fd` identifica una sesión. Cada sesión es un lease de etcd de 15 s que el cliente renueva cada 5 s con `keepalive`; al vencer se liberan sus locks (D07).
- Los identificadores binarios (`block_id` de 128 bits, `sha256`) van en hexadecimal en JSON y como bytes en protobuf. Los *block tokens* son JWT EdDSA de 10 minutos con los claims `sub` (usuario), `bid` (bloque) y `op` (`read` o `write`); el cliente guarda cada ubicación en caché mientras su token sea válido.
- Las `entries` que devuelve `lock` tienen el formato de las de `GET /sessions/{fd}/blocks`, con réplicas y token, para leer el contenido vigente antes de modificarlo.
- Los errores siguen el formato y la tabla del apartado de errores de este anexo.

Endpoints REST de `meta`

Método y ruta	Operación	Petición → respuesta	Errores
`POST /auth/login`	CU-01	`{user, password}` → `{access_token, expires_in}`	401
`GET /fs/{path}`	`stat`	→ `{type, size, version, block_size, mtime, replication}`	404
`GET /fs/{path}?list=true&after=`	`ls`	→ `{entries: [{name, type, size, version, mtime}], next}`	404
`POST /fs/{path}`	`mkdir`	`{type: "dir", parents}` → 201	404, 409
`DELETE /fs/{path}?recursive=`	`rm`, `rmdir`	→ 204	404, 409
`POST /move`	`mv`	`{source, destination}` → 200	404, 409
`POST /open`	`open`	`{path, mode, block_size?}` → `{fd, inode, version, size, block_size, lease_ttl, lock?}`	404, 409, 423
`GET /sessions/{fd}/blocks?from=&to=`	ubicaciones	→ `{blocks: [{index, block_id, size, sha256, replicas: [{dn, addr}], token}]}`	404
`POST /sessions/{fd}/allocate`	reservar bloques	`{first_index, count, exclude?}` → `{blocks: [{index, block_id, pipeline: [{dn, addr}], token}]}`	423, 503, 507
`POST /sessions/{fd}/locks`	`lock`	`{from, to}` → `{lock_id, fencing_token, entries}`	423
`DELETE /sessions/{fd}/locks/{lock_id}`	`unlock`	→ 204	404
`POST /sessions/{fd}/commit`	`flush`	ver el ejemplo → `{version, under_replicated}`	409, 422, 503
`POST /sessions/{fd}/keepalive`	renovar sesión	→ `{ttl}`	409
`DELETE /sessions/{fd}`	`close`	→ 204; libera locks y sesión (la librería hace antes el commit)	—

Método y ruta	Operación	Petición → respuesta	Errores
`POST /admin/users`, `DELETE /admin/users/{uid}`	CU-07	`{user, password, role}` → 201; → 204	403, 409
`GET /admin/nodes`	CU-07	→ `{nodes: [{dn, state, free_bytes, blocks}]}`	403

Ejemplos

Apertura para lectura y consulta de ubicaciones de los primeros bloques:

```
POST /v1/open
{ "path": "/datos/video.mp4", "mode": "r" }

200 OK
{ "fd": "7f3c2a90", "inode": "a41f77d0", "version": 6, "size": 1073741824,
  "block_size": 8388608, "lease_ttl": 15 }

GET /v1/sessions/7f3c2a90/blocks?from=0&to=1

200 OK
{ "blocks": [
  { "index": 0, "block_id": "3c9a51e2...", "size": 8388608, "sha256": "a7f1c0...",
    "replicas": [ { "dn": "dn-1", "addr": "10.0.2.11:8445" },
                  { "dn": "dn-3", "addr": "10.0.2.13:8445" },
                  { "dn": "dn-4", "addr": "10.0.2.14:8445" } ],
    "token": "eyJhbGciOiJFZERTQSJ9..." },
  { "index": 1, "block_id": "d07b6e19...", "...": "..." } ] }
```

Reserva de un bloque para escribir: `meta` devuelve la cadena de nodos que lo replicará.

```
POST /v1/sessions/7f3c2a90/allocate
{ "first_index": 3, "count": 1 }

200 OK
{ "blocks": [ { "index": 3, "block_id": "9b1e04c7...",
                "pipeline": [ { "dn": "dn-2", "addr": "10.0.2.12:8445" },
                              { "dn": "dn-4", "addr": "10.0.2.14:8445" },
                              { "dn": "dn-1", "addr": "10.0.2.11:8445" } ],
                "token": "eyJhbGciOiJFZERTQSJ9..." } ] }
```

Commit de una escritura parcial: el cliente publica el bloque 3 reescrito e informa los nodos que lo guardaron; `commit_id` lo genera el cliente y hace que reintentar sea seguro (sección 6).

```
POST /v1/sessions/7f3c2a90/commit
Authorization: Bearer eyJhbGciOiJFZERTQSJ9...

{ "commit_id": "c1d2e3f4", "fencing_token": 48213, "base_version": 6,
  "entries": [ { "index": 3, "block_id": "9b1e04c7...", "size": 8388608,
                 "sha256": "5f2a9c...", "stored": ["dn-2", "dn-4", "dn-1"] } ],
  "size": 52428800 }

200 OK
{ "version": 7, "under_replicated": [] }
```

Errores

Todos los errores REST usan el mismo cuerpo; gRPC usa el estado equivalente con el código simbólico en los metadatos.

```
423 Locked
{ "error": { "code": "LOCKED", "message": "bloques 2-5 con lock de otra sesión",
             "details": { "from": 2, "to": 5, "retry_after_ms": 4000 } } }
```

Código	HTTP	gRPC	Cuándo	Acción del cliente
`INVALID_ARGUMENT`	400	`INVALID_ARGUMENT`	Petición mal formada; offset mayor que el tamaño	Corregir la petición
`UNAUTHENTICATED`	401	`UNAUTHENTICATED`	Token ausente, inválido o vencido	Renovar el token y reintentar
`PERMISSION_DENIED`	403	`PERMISSION_DENIED`	Recurso fuera del espacio del usuario; *block token* de otro bloque u operación	Ninguna
`NOT_FOUND`	404	`NOT_FOUND`	Ruta inexistente; réplica ausente en un DN	En un DN, usar otra réplica
`ALREADY_EXISTS`, `NOT_EMPTY`	409	`ALREADY_EXISTS`, `FAILED_PRECONDITION`	La ruta ya existe; `rmdir` de un directorio con entradas	Ninguna
`CONFLICT`	409	`ABORTED`	Un bloque modificado cambió desde la versión base	Reabrir y reintentar
`STALE_LEASE`	409	`ABORTED`	Lease vencido o fencing token superado	Volver a tomar el lock y reintentar el commit; si el rango cambió, recibe `CONFLICT`
`INSUFFICIENT_REPLICAS`	422	—	Algún bloque del commit tiene menos de 2 réplicas confirmadas en nodos vivos	Reescribir ese bloque con otra cadena
`LOCKED`	423	—	El rango tiene un lock de otra sesión (D17)	La librería reintenta mientras dure `espera`
`NO_SPACE`	507	`RESOURCE_EXHAUSTED`	Sin capacidad en los DN	Ninguna
`UNAVAILABLE`	503	`UNAVAILABLE`	etcd sin quórum o menos de 2 DN vivos	Reintentar con espera exponencial
`CORRUPT`	—	`DATA_LOSS`	El hash del bloque leído no coincide	Usar otra réplica; `meta` lo re-replica

Servicios gRPC

```proto
syntax = "proto3";
package dfsha.v1;

// Plano de datos: lo expone cada DN en el puerto 8445.
service BlockService {
  // Cliente -> DN-1 -> DN-2 -> DN-3. El primer mensaje es la cabecera; el resto, fragmentos.
  rpc WriteBlock(stream WriteBlockRequest) returns (WriteBlockResponse);
  // Lectura completa o parcial; también la usa un DN para re-replicar desde otro.
  rpc ReadBlock(ReadBlockRequest) returns (stream BlockChunk);
}

message WriteBlockRequest {
  oneof msg {
    WriteHeader header = 1;
    bytes chunk = 2;                // hasta 1 MiB
  }
```

```
}
message WriteHeader {
  bytes block_id = 1;
  uint32 size = 2;
  bytes sha256 = 3;                  // del contenido sin cifrar
  string block_token = 4;            // JWT EdDSA, op = write
  repeated string pipeline = 5;      // direcciones de los DN restantes de la cadena
}
message WriteBlockResponse {
  repeated string stored = 1;        // DN que persistieron el bloque
  repeated string failed = 2;        // DN de la cadena que no confirmaron
}
message ReadBlockRequest {
  bytes block_id = 1;
  string block_token = 2;            // op = read
  uint32 offset = 3;
  uint32 length = 4;                 // 0 = hasta el final del bloque
}
message BlockChunk { bytes data = 1; }

// Plano de control: lo expone meta en el puerto 8444 para los DN (mTLS).
service ControlService {
  rpc Register(RegisterRequest) returns (RegisterResponse);
  rpc Heartbeat(HeartbeatRequest) returns (HeartbeatResponse);
  rpc ReportBlocks(stream BlockReport) returns (ReportAck);
}

message RegisterRequest {
  string dn = 1;
  string addr = 2;
  uint64 capacity_bytes = 3;
}
message RegisterResponse { uint32 heartbeat_interval_ms = 1; }

message HeartbeatRequest {
  string dn = 1;
  uint64 free_bytes = 2;
  uint32 active_streams = 3;
  repeated TaskResult results = 4;   // tareas terminadas desde el último heartbeat
}
message HeartbeatResponse { repeated Task tasks = 1; }

message Task {
  string id = 1;
  oneof kind {
    Replicate replicate = 2;         // copiar un bloque desde otra réplica
    Delete delete = 3;               // borrar bloques sin referencia
  }
}
message Replicate {
  bytes block_id = 1;
  repeated string sources = 2;       // réplicas vivas, en orden de preferencia
  string block_token = 3;            // op = read sobre las fuentes
}
message Delete { repeated bytes block_ids = 1; }
message TaskResult {
  string task_id = 1;
  bool ok = 2;
}
```

```
message BlockReport { string dn = 1; repeated bytes block_ids = 2; }  // por lotes
message ReportAck { repeated bytes unknown = 1; }  // bloques que etcd no conoce; el DN los borra pasada 1 h
```

Anexo B. Esquema de claves en etcd

Claves y valores

Los valores son JSON; las claves con lease desaparecen solas cuando este vence.

Clave	Valor	Lease
`/users/<uid>`	`{name, argon2id, role, root}`	—
`/usernames/<name>`	`uid` (búsqueda en `login`)	—
`/dirents/<ino_padre>/<nombre>`	`{ino, type}`	—
`/inodes/<ino>` (archivo)	`{type: "file", owner, parent, block_size, version, manifest, size, sha256?, mtime}`	—
`/inodes/<ino>` (directorio)	`{type: "dir", owner, parent, children, mtime}`	—
`/manifests/<id>`	`{ino, version, size, entries: [[block_id, size, sha256], …]}`	—
`/retired/<id>`	vacío: manifiesto reemplazado o borrado que aún puede leerse	1 h
`/blocks/<block_id>`	`{size, sha256, replicas: [dn, …], state, created}`; `state` es `ok`, `under` o `corrupt`	—
`/sessions/<fd>`	`{uid, ino, mode, version}`	sesión
`/locks/<ino>/<fd>/<n>`	`{from, to}`; su `create_revision` es el fencing token	sesión
`/commits/<commit_id>`	`{ino, version}`	1 h
`/trash/<ino>`	raíz de un subárbol desvinculado por `rm -r`, pendiente de GC	—
`/nodes/<dn>`	`{addr, capacity, state}`; `state` es `active` o `dead`	—
`/alive/<dn>`	`{free_bytes, active_streams}`	15 s
`/tasks/<dn>/<task_id>`	`Task` del Anexo A, en JSON	—
`/coordinator`	`{meta}`	coordinador

Transacciones

Cada operación lee lo que necesita, prepara los cambios y los aplica con una transacción que compara lo leído; si la comparación falla, reintenta o devuelve el error correspondiente. `rev(k)` es la `mod_revision` leída de la clave `k`.

Operación	Compara	Escribe
`mkdir`, crear archivo	`/dirents/<p>/<n>` no existe; `rev(/inodes/<p>)`	`/dirents/<p>/<n>`, `/inodes/<nuevo>`, `/inodes/<p>` con `children + 1`
`rmdir`	`rev(/inodes/<d>)` con `children = 0`; `rev(/dirents/<p>/<n>)`	Borra `/dirents/<p>/<n>` e `/inodes/<d>`; `/inodes/<p>` con `children - 1`
`rm`	`rev(/dirents/<p>/<n>)`; `rev(/inodes/<p>)`	Borra `/dirents/<p>/<n>`; `/inodes/<p>` con `children - 1`; `/retired/<manifest>` o, con `-r`, `/trash/<ino>`
`mv`	Destino no existe; `rev` del origen y de ambos padres	Borra el origen; escribe el destino, ambos padres y `parent` del inodo

Operación	Compara	Escribe
`open`	—	`/sessions/<fd>`; en `w` y `a`, también su lock
`lock`	—	`/locks/<ino>/<fd>/<n>`; si existe una clave anterior con rango solapado, la borra y responde `LOCKED` (D17)
`commit`, en lotes previos	—	`/blocks/<id>` de cada bloque nuevo
`commit`, transacción final	`create_revision` de cada lock = su token; `rev(/inodes/<ino>)`; `/commits/<id>` no existe	`/manifests/<nuevo>`, `/inodes/<ino>` con `version + 1`, `/retired/<anterior>`, `/commits/<id>`
Heartbeat	—	Renueva el lease de `/alive/<dn>`; lee `/tasks/<dn>/`
Órdenes del coordinador	`create_revision(/coordinator)` es la suya	`/tasks/<dn>/…`, `/blocks/<id>`

Un `mv` de un directorio verifica antes que el destino no esté dentro del propio origen. Crear o borrar entradas modifica el inodo del directorio padre, así que `rmdir` detecta en su comparación cualquier creación concurrente.

Recolección de basura

El coordinador ejecuta periódicamente un marcado y barrido (D18):

1. Marca como vivos los manifiestos vigentes (referenciados por un inodo) y los retirados, y los bloques que aparecen en ellos.
2. Borra los manifiestos no marcados.
3. Para cada bloque de `/blocks` no marcado y creado hace más de 1 h, envía órdenes `Delete` a sus réplicas y, cuando confirman, borra su clave.
4. Recorre cada subárbol en `/trash` y borra sus entradas e inodos en lotes, retirando sus manifiestos.

Los bloques subidos pero nunca confirmados no tienen clave en `/blocks`: el DN los informa en su reporte, `meta` los devuelve como desconocidos y el DN los borra cuando tienen más de 1 h.

Anexo C. Reglas de red del despliegue

Flujos admitidos por los grupos de seguridad de la VPC (sección 3.2); todo lo demás se rechaza.

Origen	Destino	Puerto	Uso
SG-cliente	SG-meta	8443/TCP	API REST (HTTPS)
SG-cliente	SG-dn	8445/TCP	Transferencia de bloques (gRPC, TLS)
SG-dn	SG-dn	8445/TCP	Replicación en cadena y re-replicación (gRPC, mTLS)
SG-dn	SG-meta	8444/TCP	Heartbeats, reportes y órdenes (gRPC, mTLS)
SG-meta	SG-meta	2379–2380/TCP	etcd: clientes y pares (mTLS)

Anexo D. Registro de decisiones

Decisiones citadas en el texto como Dnn; cada una se justifica en la sección correspondiente.

ID	Decisión	Estado	Requisitos
D01	Plano de control: 3 nodos `meta` sin estado sobre etcd (Raft)	Aceptada	RNF2, RNF3

ID	Decisión	Estado	Requisitos
D02	Coordinador elegido entre los `meta` para tareas de fondo	Aceptada	RNF2
D03	Bloques inmutables + manifiesto con commit atómico (copy-on-write)	Aceptada	RNF3, RF3
D04	Tamaño de bloque fijo de 8 MiB, configurable al crear el archivo	Aceptada	RNF1, RNF4, RNF5
D05	Replicación R = 3 en cadena, confirmación con W ≥ 2	Aceptada	RNF2, RNF3
D06	Semántica close-to-open; write() completo; snapshot por open()	Aceptada	RF3, RNF3
D07	lock() por rango de bloques con lease + fencing token; commit CAS	Aceptada	RF3, RNF3, RNF5
D08	REST/JSON para control, gRPC streaming para datos, sin MOM	Aceptada	RNF5
D09	Descubrimiento por nombre DNS lógico + ubicaciones desde metadatos	Aceptada	RNF7
D10	JWT, block tokens, TLS 1.3/mTLS, AES-256-GCM en servidor	Aceptada	RNF6
D11	Heartbeats 5 s / sospechoso 15 s / caído 60 s; re-replicación	Aceptada	RNF2
D12	Metadatos CP; datos disponibles mientras quede una réplica	Aceptada	RNF2, RNF3
D13	Manifiesto completo en un solo valor por versión	Aceptada	RNF1, RNF3
D14	Python con FastAPI, grpcio y etcd3gw para la implementación	Aceptada	RNF5, RNF6
D15	Espacio de nombres por inodos; raíz propia por usuario	Aceptada	RF1, RNF6
D16	Ubicaciones en etcd a partir de las confirmaciones de la cadena	Aceptada	RNF2, RNF3, RNF5
D17	Conflicto de lock: `LOCKED` inmediato; reintento en el cliente	Aceptada	RF3, RNF5
D18	Retención: sin historia; manifiestos retirados 1 h; GC con gracia	Aceptada	RNF1, RNF3
D19	Locks implícitos con refresco; reintento ante fallo en la cadena	Aceptada	RF3, RNF2, RNF5
D20	Escala objetivo: ~10 DN, ~100 usuarios, archivos de hasta 10 GiB	Aceptada	RNF1

Anexo E. Secuencias de lectura, escritura parcial y borrado

Complementan la secuencia de carga de la sección 2. Las flechas punteadas son respuestas.

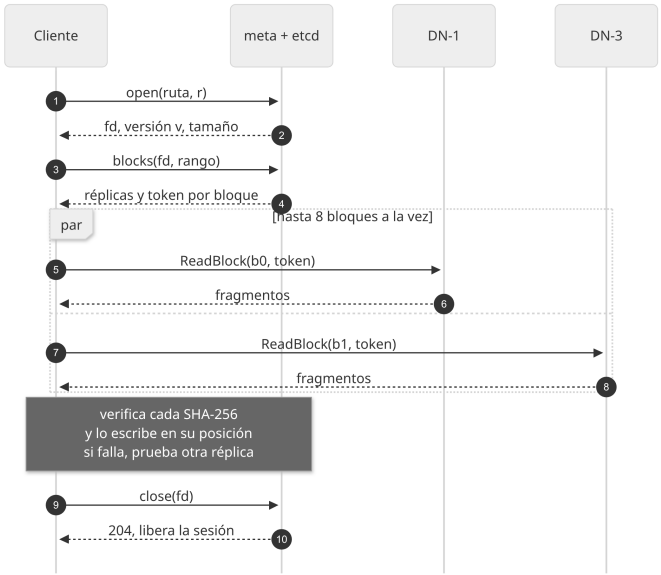


Figura 5. *Lectura (`get`, u `open(r)` + `read`): la versión queda fija al abrir y cada bloque se lee de una sola réplica (secciones 4 y 6).*

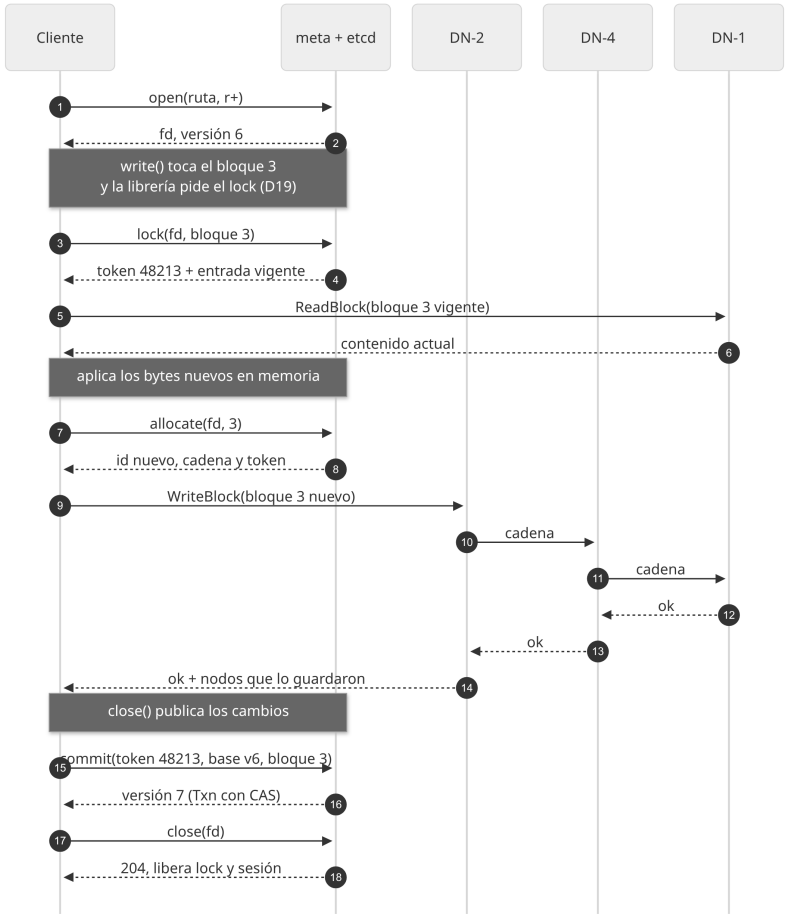


Figura 6. *Escritura parcial en `r+`: el bloque afectado se reescribe como uno nuevo y la versión cambia con un único commit (secciones 6 y 8).*

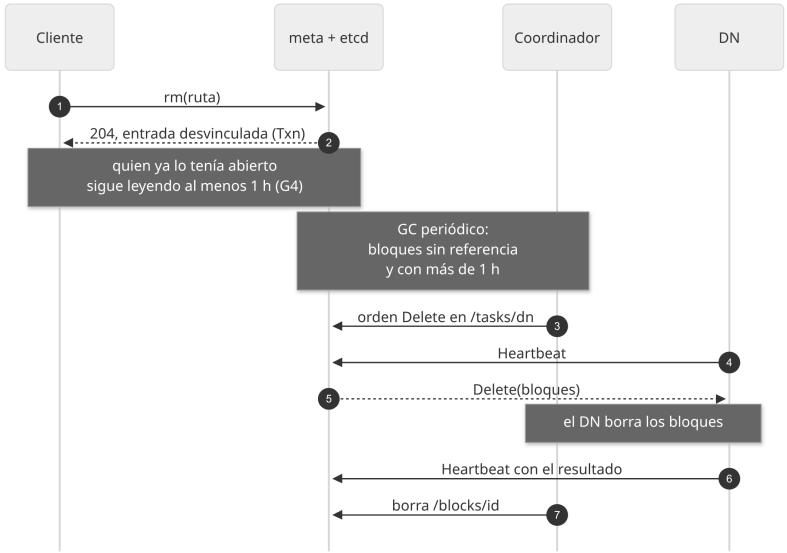


Figura 7. *Borrado: la desvinculación es inmediata y el espacio se libera después, con el GC y las órdenes en los heartbeats (Anexo B).*